Finite State Machine LED Controller

ARM Assembly on Microbit v2 (nRF52833)

Computer Architecture Assignment — Project Report

Name	Roll Number
Charan	BC2025032
Srikar	BC2025074
Sathvik	BC2025087
Lokesh	BC2025023

Department of Computer Science & Engineering

1. Project Overview

For this assignment, we chose the Finite State Machine (FSM) LED Controller project. The goal was to implement an FSM in ARM Assembly that controls the 5×5 LED matrix on the Microbit v2, with button presses used to switch between different visual states.

We implemented 8 states — all static patterns. Button A navigates to the previous state and Button B navigates to the next state, wrapping around at both ends. Everything is done through direct memory-mapped I/O with no libraries or abstractions.

1.1 States Overview

State	Pattern	Description
0	Plus	Static cross / plus sign
1	Cross (X)	Static diagonal X pattern
2	Diamond	Static diamond outline
3	Checkerboard	Alternating lit/unlit grid
4	Pyramid	Triangle shape, bottom-aligned
5	Smiley Face	Static smiley emoji
6	House	House shape with door gap
7	Heart	Heart shape pattern

2. Architecture Concepts

2.1 Finite State Machine (FSM)

The core of the project is a software FSM. The current state is stored in register R6 (values 0–7). When a button is pressed, R6 is incremented or decremented with wrap-around. The main loop reads R6 and renders the corresponding pattern each iteration.

2.2 Memory-Mapped I/O

The nRF52833 uses memory-mapped I/O, meaning every GPIO operation is a load or store to a specific address. No HAL or OS is involved — just LDR/STR directly to hardware registers.

Register	Address / Offset	Purpose
P0 Base	0x50000000	GPIO port 0 base
P1 Base	0x50000300	GPIO port 1 base
OUTSET	Base + 0x508	Set pins HIGH

OUTCLR	Base + 0x50C	Set pins LOW
IN	Base + 0x510	Read pin states
DIRSET	Base + 0x518	Configure pins as output
PIN_CNF[n]	Base + 0x700 + n×4	Configure individual pin

2.3 LED Matrix Multiplexing

The 5×5 LED matrix uses only 10 GPIO pins through multiplexing — 5 row pins and 5 column pins. To light a specific LED, the row pin is set HIGH (anode) and the column pin is set LOW (cathode). Only one row is active at a time, but since all 5 rows are scanned fast enough, persistence of vision makes the full image appear lit.

Pin mapping: COL1=P0.28, COL2=P0.11, COL3=P0.31, COL4=P1.05, COL5=P0.30. ROW1=P0.21, ROW2=P0.22, ROW3=P0.15, ROW4=P0.24, ROW5=P0.19. Note that COL4 is on P1 (a different GPIO port), which requires a separate base address register.

2.4 Stack Operations

Since draw_row calls clear_leds internally, it is a non-leaf function — LR gets overwritten if not saved. We use PUSH {LR} at the entry of draw_row and POP {PC} at the exit, restoring the program counter directly from the stack. All working registers used inside the function are also saved and restored to prevent corruption of the caller's state.

2.5 Bit Manipulation

Each row of a pattern is stored as a single byte, where bits 0–4 represent columns 1–5. The draw_row function uses the TST instruction to test each bit and conditionally activates the corresponding column GPIO pin. This demonstrates hardware-level bit manipulation directly in assembly.

3. Implementation Details

3.1 Data Layout

All patterns are stored in the .data section as byte arrays, 5 bytes per pattern (one byte per row). A pointer table in the .data section holds the address of each pattern, allowing the main loop to index into it using R6 × 4.

Pattern byte encoding — bit positions map to columns as follows:

```
bit 0 = COL1, bit 1 = COL2, bit 2 = COL3, bit 3 = COL4, bit 4 = COL5
```

Example patterns and their byte values:

Pattern	Row 1	Row 2	Row 3	Row 4	Row 5
Plus	0x04	0x04	0x1F	0x04	0x04
Cross	0x11	0x0A	0x04	0x0A	0x11
Diamond	0x04	0x0A	0x11	0x0A	0x04
Checker	0x0A	0x15	0x0A	0x15	0x0A
Pyramid	0x00	0x04	0x0E	0x1F	0x00
Smiley	0x0A	0x0A	0x00	0x11	0x0E
House	0x04	0x0E	0x1F	0x1B	0x1B
Heart	0x0A	0x1F	0x0E	0x04	0x00

3.2 Main Loop Structure

The main loop performs one complete 5-row scan of the current pattern on every iteration, then polls both button pins. If neither button is pressed the loop repeats immediately, keeping the display continuously refreshed. This scan-then-poll approach ensures the display never goes blank while waiting for input.

3.3 Button Handling and Debouncing

Both buttons are active-LOW — pressing them pulls the pin to GND. The main loop reads P0's IN register and tests the relevant bits using TST. On a press, the state index R6 is incremented or decremented, then the following debounce sequence runs:

1. A ~100,000-cycle busy-wait loop (delay_debounce) runs to let the signal settle.
2. The code waits in wait_release until both button pins read HIGH (released).
3. One more delay_debounce runs before resuming the main loop.

This prevents a single press from triggering multiple state transitions, since the polling loop runs far faster than a human can mechanically press and release a button.

3.4 COL4 Handling (GPIO Port 1)

COL4 is mapped to P1.05, which is on a completely different GPIO port (base 0x50000300) from the other nine LED pins. The draw_row and clear_leds functions maintain a second base address in register R1 and issue separate OUTSET/OUTCLR stores to R1 whenever COL4 needs to be driven. This is the most hardware-specific part of the implementation.

4. Challenges Faced

4.1 COL4 Being on P1

COL4 (P1.05) is on a different GPIO port than the other columns. Early on, any pattern using column 4 simply did not light up. The fix required keeping a second base address register (R1 = 0x50000300) and routing all COL4 OUTSET/OUTCLR writes there instead of to R0. Debugging this took considerable time because visually it looked like a bitmask error rather than an address error.

4.2 LR Getting Clobbered

The first version of draw_row only saved LR. But since draw_row calls clear_leds internally, that BL overwrote LR again before the POP. Patterns were rendering incorrectly and the root cause was non-obvious. The fix was to push all registers used inside the function at entry and pop them (restoring PC) at exit.

4.3 Button Bouncing

Without debouncing, a single button press would skip through 2 or 3 states at once because the polling loop cycles far faster than the button contact settles electrically. Adding the delay_debounce loop and the wait_release loop eliminated this completely.

4.4 GPIO Immediate Offset Limitation

ARM Thumb's STR instruction only allows a 5-bit unsigned immediate offset (0–124). The GPIO register offsets 0x508 (OUTSET) and 0x50C (OUTCLR) far exceed this limit. Attempting to use them as immediates produces incorrect machine code. The correct approach is to load the full absolute address into a register and store with a zero offset: LDR R2, =0x5000050C followed by STR Rx, [R2].

5. Results

All eight patterns display correctly and stably on the 5×5 LED matrix. Button A and Button B cycle through states reliably with no missed or double transitions. The display remains continuously lit with no flicker during normal operation.

The project demonstrated several core computer architecture concepts in a tangible, hands-on way: finite state machine implementation in assembly, memory-mapped I/O, LED matrix multiplexing with persistence of vision, and strict stack discipline in non-leaf subroutines.

5.1 Possible Future Improvements

- Use hardware timer interrupts instead of busy-wait delays for more accurate scan timing.
- Add PWM on the row pins to control per-LED brightness.
- Read the on-board accelerometer over I2C and use tilt gestures to trigger state changes.
- Add a scrolling text mode using a character font table stored in flash memory.
- Save the current state to non-volatile memory so it persists across power cycles.
- Introduce animated patterns (e.g., a beating heart or expanding ripple) as additional states.

— End of Report —